PRACTICAL-3

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <algorithm>

using namespace std;
using namespace chrono;

//------------------------------------------------
// Function to maintain Max Heap property
//------------------------------------------------
void maxHeapify(vector<int> &arr, int size, int root)
{
   int largest = root;
   int leftChild = 2 * root + 1;
   int rightChild = 2 * root + 2;

   // Check if left child is greater
   if (leftChild < size && arr[leftChild] > arr[largest])
   {
      largest = leftChild;
   }

   // Check if right child is greater
   if (rightChild < size && arr[rightChild] > arr[largest])
   {
      largest = rightChild;
   }
```

PRACTICAL-3

```
    // If root is not the largest, swap and heapify again
    if (largest != root)
    {
        swap(arr[root], arr[largest]);
        maxHeapify(arr, size, largest);
    }
}

//------------------------------------------------
// Max Heap Sort Function
//------------------------------------------------
void maxHeapSort(vector<int> &arr)
{
    int size = arr.size();

    // Step 1: Build Max Heap
    for (int i = size / 2 - 1; i >= 0; i--)
    {
        maxHeapify(arr, size, i);
    }

    // Step 2: Extract largest element one by one
    for (int i = size - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);

        // Restore heap property
        maxHeapify(arr, i, 0);
    }
}
```

PRACTICAL-3

```
//-------------------------------------------------
// Function to maintain Min Heap property
//-------------------------------------------------
void minHeapify(vector<int> &arr, int size, int root)
{
   int smallest = root;
   int leftChild = 2 * root + 1;
   int rightChild = 2 * root + 2;

   // Check left child
   if (leftChild < size && arr[leftChild] < arr[smallest])
   {
      smallest = leftChild;
   }

   // Check right child
   if (rightChild < size && arr[rightChild] < arr[smallest])
   {
      smallest = rightChild;
   }

   // Swap if root is not smallest
   if (smallest != root)
   {
      swap(arr[root], arr[smallest]);
      minHeapify(arr, size, smallest);
   }
}

//-------------------------------------------------
// Min Heap Sort Function
```

PRACTICAL-3

```
//-------------------------------------------------
void minHeapSort(vector<int> &arr)
{
   int size = arr.size();

   // Step 1: Build Min Heap
   for (int i = size / 2 - 1; i >= 0; i--)
   {
      minHeapify(arr, size, i);
   }

   // Step 2: Extract smallest element one by one
   for (int i = size - 1; i > 0; i--)
   {
      swap(arr[0], arr[i]);

      // Restore heap property
      minHeapify(arr, i, 0);
   }

   // Reverse array to get ascending order
   reverse(arr.begin(), arr.end());
}

//-------------------------------------------------
// Main Function
//-------------------------------------------------
int main()
{
   int numberOfElements;
```

PRACTICAL-3

```
cout << "Enter number of elements: ";
cin >> numberOfElements;

// Create vector
vector<int> originalArray(numberOfElements);

// Generate random numbers
srand(time(0));

for (int i = 0; i < numberOfElements; i++)
{
   originalArray[i] = rand() % 100000;
}

// Create copies of original array
vector<int> maxHeapArray = originalArray;
vector<int> minHeapArray = originalArray;

//---------------- MAX HEAP SORT ----------------//

auto startMax = high_resolution_clock::now();

maxHeapSort(maxHeapArray);

auto endMax = high_resolution_clock::now();

//---------------- MIN HEAP SORT ----------------//

auto startMin = high_resolution_clock::now();

minHeapSort(minHeapArray);
```

PRACTICAL-3

```
auto endMin = high_resolution_clock::now();

//---------------- Calculate Time ----------------//

auto maxNano =
    duration_cast<nanoseconds>(endMax - startMax);

auto maxMicro =
    duration_cast<microseconds>(endMax - startMax);

auto minNano =
    duration_cast<nanoseconds>(endMin - startMin);

auto minMicro =
    duration_cast<microseconds>(endMin - startMin);

//---------------- Display Results ----------------//

cout << "\n========== MAX HEAP SORT ==========\n";
cout << "Execution Time (Nanoseconds)  : "
     << maxNano.count() << " ns" << endl;

cout << "Execution Time (Microseconds) : "
     << maxMicro.count() << " us" << endl;

cout << "\n========== MIN HEAP SORT ==========\n";
cout << "Execution Time (Nanoseconds)  : "
     << minNano.count() << " ns" << endl;

cout << "Execution Time (Microseconds) : "
```

PRACTICAL-3

```
        << minMicro.count() << " us" << endl;


    return 0;
}
```